Sieve: Cache-Aware, Segment-Level Routing Across Compression, KV-Prefix Reuse and Externalization for Local Large Language Model Prompts

[Author Name]*
*[Department], [Institution], [City, Country]
Corresponding author: [Author Name] (e-mail: [address]).

***Abstract*—Local large language models share one fixed context window between the prompt and the answer, and long conversations waste it on verbose, repeated and stale text. Prompt compression reduces tokens but rewrites the prefix, which defeats the key-value (KV) prefix cache that local servers rely on. This paper presents Sieve, a middleware that splits each request into segments and routes every segment to one of three lanes: COMPRESS (LLMLingua-2 pruning at a per-segment ratio, checked by a rule-based verifier of facts and their context), CACHE (kept byte-identical so the KV prefix is reused) or EXTERNALIZE (replaced by a deterministic stub that the model can expand through a tool call). Hard rules run first, a cost-sensitive gradient-boosting classifier routes the rest, and a per-session ledger replays emitted strings so compression does not invalidate the cached prefix. On 180 ground-truth questions from 76 synthetic conversations with invented entities, run on Qwen3-14B through Ollama with a 4096-token window, Sieve reduced prompt tokens by 31.0% (95% CI 27.8–34.0%) and raised mean output headroom from 1,349 to 2,393 tokens. Its accuracy change of +5.0 percentage points (CI -0.6 to +11.1) is not statistically distinguishable from zero, while on prompts that overflowed the window accuracy rose by 36.2 points (CI +19.1 to +53.2). Median time to first token on cold questions fell by 1.12 s. A controlled test shows that re-compressing a prefix reduces prefill reuse from 95% to 2%, and that sending the memoized compressed string restores it to 89%. Code, logs and traces are public.**

***Index Terms*—Large language models, prompt compression, key-value cache, context management, local inference, cost-sensitive classification, retrieval on demand.**

I. Introduction

LARGE language models (LLMs) are increasingly run on local hardware for privacy, cost and latency reasons [1], [2]. A local deployment exposes one hard resource: the context window, here $n_{\text{ctx}} = 4096$ tokens, shared by the system prompt, the conversation history, pasted documents and tool output, and the answer itself [3], [4]. Long sessions fill it in three distinct ways. Some text is needed but verbose; some is static and resent on every turn; and some is bulky but rarely relevant, such as an old document or a large tool dump, a pattern that is common when LLMs are embedded in engineering workflows [5]. When the raw prompt exceeds the window the server silently truncates it, and facts disappear.

Each kind of waste has a known remedy, but the remedies interfere. Token-level prompt compression, for example Selective Context [6], LLMLingua [7], LongLLMLingua [8] and LLMLingua-2 [9], shortens verbose text. Key-value (KV) prefix caching, as in PagedAttention [10], Prompt Cache [11] and SGLang [12], makes static prefixes nearly free to re-process. Retrieval on demand, as in memory-managed agents [13], [14] and tool-using models [15], [16], keeps rarely needed text out of the prompt. However, compression rewrites the prompt, and any change early in the prompt forces the server to prefill every later token again. A compressor that is re-run on every turn can therefore make warm turns slower even as it makes prompts shorter.

This paper studies whether a *per-segment* routing policy that chooses among compression, caching and externalization, combined with a mechanism that keeps emitted text byte-stable across turns, reduces prompt tokens and increases output headroom without losing accuracy. The contributions are:

- **Sieve**, a three-lane middleware (COMPRESS, CACHE, EXTERNALIZE) with a rule layer, a cost-sensitive gradient-boosting router, a fact verifier that checks both atoms and the relation words around them, a session store with deterministic stubs and an `expand_context` tool, and a prefix ledger with budget-driven epochs (Section III).
- A formulation of budget-constrained lane assignment as a multiple-choice knapsack problem, with a greedy priority allocator and a complexity analysis of every pipeline stage.
- A **controlled measurement of the cache–compression conflict** on a production local server, including a timing-based estimator of prefix reuse for servers whose token counters do not reveal cache hits.
- A matched-budget benchmark of seven conditions and five ablations on 180 ground-truth questions, with paired bootstrap statistics, per-call traces and an automatically generated list of supported and unsupported claims. Code, logs and traces are public [17].

Section II reviews related work, Section III presents Sieve, Section IV describes the experimental methodology, Section V reports the results, Section VI discusses them and their limits, and Section VII concludes.

II. Related Work

A. Large Language Models and Prompting

Pre-trained language models evolved from task-specific encoders into general LLMs; surveys trace this path [18]–[20] and the architectures and training regimes of current LLMs [3],

[4], [21], [22]. Prompting became the main interface to these models [23], [24], and evaluation practice has had to follow [25]. We use Qwen3 [26], an open-weight model family that can be served locally and supports tool calls with or without explicit reasoning. These works establish that prompt content drives model behaviour. They do not address how to fit a growing conversation into a fixed window at minimum cost.

B. Efficient Models and Inference

One line of work lowers the cost per token. Efficient transformer variants change the attention mechanism [27], [28], and broader surveys cover efficient methods across the NLP pipeline [29]. Model compression shrinks the network itself through quantization, pruning and distillation [30]–[33], which is essential on edge devices [1] and motivated by the energy cost of large models [2]. Sieve is complementary: it leaves the model unchanged and reduces the number of tokens the model must process.

C. Prompt and Context Compression

Hard-prompt methods delete tokens. Selective Context removes content with low self-information [6]; LLMLingua uses a small language model with a budget controller that assigns different ratios to prompt components [7]; LongLLMLingua adds question-aware compression and document reordering [8]; LLMLingua-2 casts compression as token classification with a distilled encoder [9]; and reinforcement learning has been used to learn discrete edit policies [34]. Soft-prompt methods compress into learned vectors (gist tokens [35], AutoCompressors [36]) and therefore require changes to the target model, while RECOMP compresses retrieved documents with trained extractive and abstractive compressors [37]. Extractive summarization offers a further family of length-reduction techniques [38]. All of these apply a single policy to the prompt or to one component of it, and none accounts for the effect of compression on the server's prefix cache.

D. KV-Cache Reuse and Serving

Serving systems reuse attention state across requests. PagedAttention manages the KV cache in pages and shares prefixes [10]; Prompt Cache precomputes attention for reusable prompt modules [11]; SGLang's RadixAttention reuses prefixes across calls of structured programs [12]; and a recent survey organizes serving optimizations from algorithms to systems [39]. The llama.cpp engine [40] used by Ollama [41] reuses the longest common prefix with earlier requests. These systems make an unchanged prefix cheap; Sieve shapes prompts so that the prefix stays unchanged when compression is used.

E. Retrieval, External Memory and Tools

Retrieval-augmented generation fetches relevant text at query time [42], [43], typically with dense retrievers [44] built on sentence encoders such as Sentence-BERT [45] and MiniLM [46]. Agent architectures keep long-term memory outside the prompt [14], [47], [48]; MemGPT pages context in and out through function calls [13]. Toolformer showed that models can learn when to call tools [15], and tool learning has since been surveyed broadly [16]. Sieve's EXTERNALIZE lane is a deterministic, session-scoped instance of this idea, evaluated against cheaper alternatives under one budget.

TABLE I
CAPABILITIES OF REPRESENTATIVE APPROACHES (✓ YES, ○ PARTIAL, ✗ NO). "TARGET UNCHANGED" MEANS THE SERVED LLM NEEDS NO TRAINING OR MODIFICATION.

Approach	Compress	Cache-aware	External.	Per-segment	Target unch.	Fact check
Selective Context [6]	✓	✗	✗	✗	✓	✗
LLMLingua [7]	✓	✗	✗	○	✓	✗
LongLLMLingua [8]	✓	✗	✗	○	✓	✗
LLMLingua-2 [9]	✓	✗	✗	✗	✓	✗
Discrete RL [34]	✓	✗	✗	✗	✓	✗
Gist / AutoCompressor [35], [36]	✓	✗	✗	✗	✗	✗
RECOMP [37]	✓	✗	✗	○	✓	✗
PagedAttention / Prompt Cache / SGLang [10]–[12]	✗	✓	✗	✗	✓	✗
MemGPT [13]	○	✗	✓	✗	✓	✗
Sieve (this work)	✓	✓	✓	✓	✓	✓

F. Long Contexts and Faithfulness

Models do not use long contexts uniformly: information in the middle of the prompt is used less reliably than information at the ends [49]. Generated answers can also be unfaithful to their input [50], [51]. Both findings shape our design: Sieve keeps chat turns in their original order, checks compressed text for lost facts, and our benchmark includes needle-position, late-recall and unanswerable questions.

G. Learning and Statistical Methods

Sieve's router is a gradient-boosting classifier [52] with a random-forest ratio regressor [53], implemented with scikit-learn [54]; class weights address the rarity of the CACHE lane [55]. Results are compared with paired bootstrap intervals [56], following established advice on comparing learning systems [57].

H. Research Gap

Table I positions Sieve against representative methods. Prior work covers one or two lanes, uses one policy per prompt or component, and does not control the interaction between compression and prefix caching. Sieve combines all three lanes under a per-segment policy, verifies compressed facts, and keeps its output byte-stable.

III. Proposed Method: Sieve

A. Problem Formulation

A request is a chat history H and a question q. Sieve splits $H \cup \{q\}$ into segments $s_1, \ldots, s_m$ of t_i tokens and assigns each a lane $\ell_i \in \{\mathrm{C}, \mathrm{K}, \mathrm{X}\}$ (compress, cache, externalize) and, for C, a keep-ratio $r_i \in \mathcal{R} = \{0.3, 0.5, 0.7, 0.9, 1\}$. The emitted length is

$$L_i(\ell_i, r_i) = \begin{cases} |\mathrm{LLMLingua2}(s_i, r_i)| & \ell_i = \mathrm{C} \\ t_i \text{ (or a back-reference)} & \ell_i = \mathrm{K} \\ |\mathrm{stub}(s_i)| & \ell_i = \mathrm{X} \end{cases} \quad (1)$$

and the prompt length is $T = \sum_i L_i + O$, where O counts every overhead Sieve adds (chat template, system note, tool definition, stubs and tail hint). Sieve seeks

$$\min_{\{\ell_i, r_i\}} T \quad \text{s.t.} \quad T \le B, \qquad \Pr[\text{needed fact lost}] \text{ small}, \tag{2}$$

with budget $B = \min(\beta\, n_{\text{ctx}},\, n_{\text{ctx}} - R)$, $\beta = 0.8$ and output reserve $R = 256$ tokens. Each segment chooses one option from a small set, so the budget constraint makes this a multiple-choice knapsack problem; Sieve solves it greedily (Section III-I). Which segments are "needed" is unknown at routing time, because future questions are unknown, so the router minimizes an expected cost instead of a hard constraint.

B. Architecture

Fig. 1 shows the pipeline. It is model-agnostic middleware between a chat client and an unmodified local server.

C. Canonicalization, Segmentation and Features

Tool output is first normalized losslessly where possible: JSON is minified, null and empty fields are dropped and identical records collapsed with a count; consecutive identical log lines become one line with a multiplicity; duplicate CSV rows are counted. Each message is then split on paragraph boundaries into chunks of 150–400 tokens. A segment's identifier is the first 12 hex digits of the SHA-256 of its text, so identical text always maps to the same identifier. Exact repeats are found by identifier and near-duplicates by 5-word shingle Jaccard similarity ≥ 0.85, both per session. Each segment receives 16 features: six core features (system flag, last-turn flag, normalized length, recency in turns, repeat count, and cosine similarity to the current question computed with MiniLM [46] on the CPU), seven kind indicators, digit density, near-duplicate score and prior expansion count.

D. Rule Layer

Rules are checked first, in priority order: (1) the current question is never altered; (2) the system prompt is cached; (3) user tags are honored (*never_compress* and *pinned* force CACHE; *never_externalize* forbids EXTERNALIZE); (4) an exact repeat of an emitted segment becomes a one-line back-reference, and a near-duplicate does so only if every fact-like atom it contains already appears in the original, so a changed value is never hidden; (5) segments under 40 tokens pass through; (6) a segment whose stub would exceed half its size may not be externalized. Rules are user-extensible from configuration.

E. Cost-Sensitive Router

For the remaining segments a gradient-boosting classifier (150 trees, depth 3) [52] estimates lane probabilities $p \in \Delta^3$, trained with balanced sample weights [55]. Sieve does not take the arg-max. It chooses

$$\hat{\ell} = \arg\min_j \sum_i p_i\, C_{ij}, \qquad C_{\text{c,x}} = 5,\ C_{ij} = 1\ (i \ne j),\ C_{ii} = 0, \tag{3}$$

so hiding a segment that should have been sent costs 5 times more than any other mistake. If $\max_i p_i < 0.55$ the segment falls back to COMPRESS at $r = 0.9$, never to EXTERNALIZE. A random forest [53] predicts the keep-ratio of COMPRESS segments; the prediction is snapped to $\mathcal{R}$ and floored per kind (documents 0.3, chat 0.4, instructions 0.9).

F. COMPRESS Lane and Fact Verifier

Compression uses LLMLingua-2 [9] on the CPU. Results are memoized on (content hash, ratio, compressor version), so the same input always produces the same string. Before compression, the verifier extracts atoms A: numbers, dates, versions, identifiers, quoted strings and capitalized entities. After compression it computes

$$\text{AR} = \frac{|\{a \in A : a \text{ in output}\}|}{|A|}, \qquad \text{CR} = \min_{a \in A} \frac{|W_a \cap N_a|}{|W_a|}, \tag{4}$$

where W_a is the set of content words of the sentence containing a (excluding atom words) and N_a the words within ± 12 positions of a in the output. Compression is accepted if every numeric or identifier atom survives, AR ≥ 0.9 and CR ≥ 0.5; otherwise the ratio is raised one bucket, up to 3 times, and finally the segment passes through uncompressed. The context-recall term was added after LLMLingua-2 was observed to keep a person's name while deleting the role that made it an answer; atom recall alone did not catch this. A repair step undoes tokenizer artifacts of the compressor (for example "5 585" → "5,585") for atoms present in the original text only.

G. CACHE and EXTERNALIZE Lanes

CACHE emits the segment unchanged. EXTERNALIZE stores the full text in a session-scoped dictionary and emits a deterministic stub `[ref:ID | kind | Nt | "summary" | keys: … | use expand_context("ID")]`, whose summary is extractive (leading sentence plus top TF-IDF terms) and whose keys are up to five salient atoms. When all chunks of a message are externalized they share one stub. The model can call `expand_context` (at most 2 rounds per request, 1,200 tokens per request); unknown identifiers return an explicit error rather than empty text, so the model cannot be misled into inventing content. With reasoning disabled, the model rarely called the tool on its own, so Sieve appends one line to the final user message listing up to six stub identifiers with their keys. Prefetching expands stubs whose similarity to the question is at least 0.75. Both the hint and prefetched text sit in the final message, after the cached prefix.

H. Prefix Ledger and Epochs

The ledger stores the exact string emitted for every segment (Algorithm 1). On later turns it replays those strings and routes only new segments, so the prompt grows by appending and the cached prefix remains valid. Old segments are not re-scored, even though their similarity to the new question has changed. A full re-route, an *epoch*, happens only when the projected prompt plus the output reserve would exceed 80% of n_{ctx}; the allocator then targets 85% of that threshold so that the

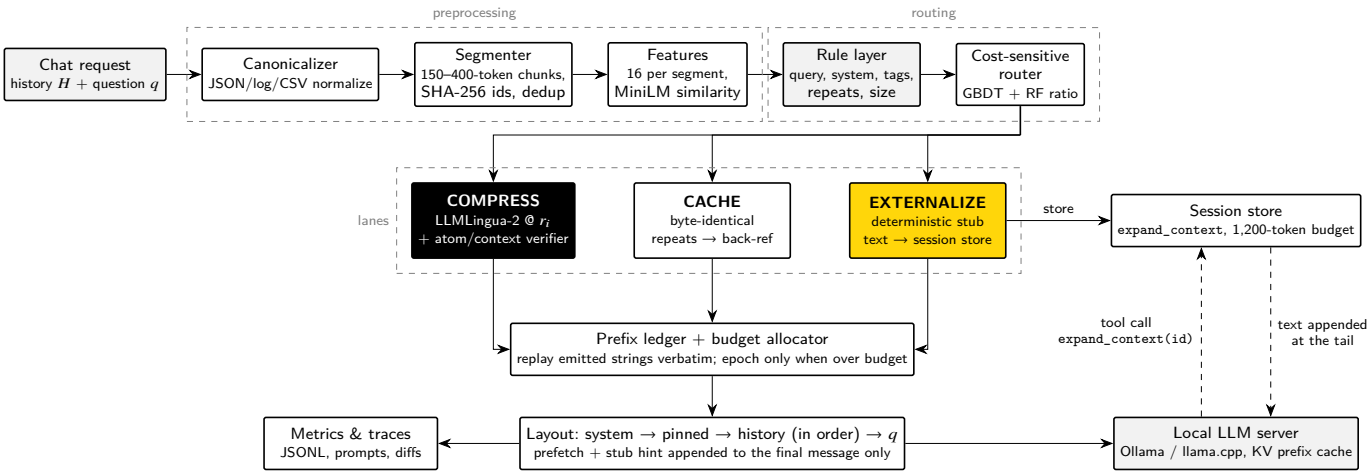


Fig. 1. Architecture of Sieve. Each segment takes exactly one lane. Rules run before the classifier and always win. The ledger replays strings emitted on earlier turns byte-for-byte so the server's KV prefix stays valid; dashed edges show the tool round trip that returns externalized text. All Sieve components run on the CPU.

Algorithm 1 Per-turn prompt assembly with the prefix ledger

Require: history H, question q, ledger $\mathcal{L}$ (segment key $\mapsto$ emitted string), store S

$G \leftarrow \textsc{Segment}(H \cup \{q\})$; compute features of G w.r.t. q
if $\mathcal{L} = \emptyset$ **then**
  $E \leftarrow \textsc{RouteAll}(G)$; *epoch* $\leftarrow$ true
else
  $E \leftarrow \{g \mapsto \mathcal{L}[g] : g \in G \cap \mathcal{L}\}$ ▷ replay verbatim
  $E \leftarrow E \cup \textsc{Route}(G \setminus \mathcal{L})$
  if $|E| + \text{hint} + R > 0.8\, n_{\text{ctx}}$ **then**
    $E \leftarrow \textsc{RouteAll}(G)$; *epoch* $\leftarrow$ true
  end if
end if
if *epoch* and $|E| + \text{hint} + R > 0.8\, n_{\text{ctx}}$ **then**
  $E \leftarrow \textsc{Allocate}(E,\ 0.85 \cdot (0.8\, n_{\text{ctx}} - R))$
end if
$P \leftarrow \textsc{Layout}(E)$ + prefetch + stub hint ▷ tail only
$\mathcal{L} \leftarrow E$; **return** P

Algorithm 2 Greedy budget allocation

Require: emitted segments E, target τ, hard limit B

$Q \leftarrow$ unprotected segments sorted by (similarity to q, turn, position) ascending
for $s \in Q$ **while** $|E| > \tau$ **do** lower r_s to its kind floor; recompress and verify
end for
for $s \in Q$ **while** $|E| > \tau$ **do**
  if EXTERNALIZE allowed for s **then** externalize s
  end if
end for
for $s \in Q$ by age **while** $|E| > B$ **do** drop s; flag hard truncation
end for

next turns do not immediately trigger another epoch. The tool definition is attached the first time anything is externalized and then kept, because the server renders tools inside the system block and toggling them would invalidate the whole prefix.

I. Budget Allocator

Algorithm 2 tightens the least relevant and oldest segments first: it lowers ratios, then externalizes, and drops segments only as a last resort, which is logged as a hard truncation. Protected segments (question, system prompt, pinned and *never_compress* text, back-references) are never touched. After allocation the prompt is rendered and counted exactly; if it still exceeds B, allocation repeats against the exact count.

J. Complexity

For a request with n tokens in m segments: canonicalization, tokenization and chunking are $O(n)$; exact-duplicate detection is $O(m)$ with a hash map; near-duplicate detection compares shingle sets pairwise, $O(m^2 k)$ for k shingles per segment; features cost $O(m)$ encoder passes of bounded length; rule checks are $O(m)$; routing is $O(m\,T\,d)$ for T trees of depth d; ledger replay is $O(m)$ dictionary lookups; and the allocator sorts once, $O(m \log m)$, then makes at most two linear passes, each step re-compressing one segment of at most 400 tokens. Compression and embedding results are memoized, so a replayed turn costs only the new segments. Measured CPU overhead is reported in Section V.

IV. Experimental Methodology

A. Research Questions

- **RQ1** Does Sieve reduce prompt tokens and increase output headroom without lowering accuracy?
- **RQ2** How does it affect time to first token on cold and warm questions?
- **RQ3** Does compression invalidate the server's prefix cache, and does the ledger prevent it?
- **RQ4** Which components matter (verifier, ledger, router, prefetch)?
- **RQ5** How accurate is the router on conversations it has not seen, and when does externalization stop paying off?

B. Platform

All runs used Qwen3-14B [26] at Q4_K_M quantization (14.8B parameters) served by Ollama 0.32.15 [41] on an Apple M5 Pro with 24 GB of unified memory; the server reported the

TABLE II
WORKLOAD FAMILIES (QUESTIONS PER FAMILY IN PARENTHESES).

ID	What the family stresses
A	Long system prompt, short chat (19)
B	Long chat, old irrelevant history (20)
C	Verbose document Q&A (27)
D	Same document re-pasted (18)
E	Needle in a haystack (10)
F	Topic switches, late recall (16)
G	Number / ID dense text (16)
H	Bloated tool output (6)
I	Code Q&A (8)
J	Adversarial edge cases (8)
K	Mixed 10–20 turn sessions (32)

model fully resident on the GPU (100% of 8.97 GiB). Requests used $n_{\text{ctx}} = 4096$, at most 64 output tokens, temperature 0, a fixed seed and reasoning disabled; the selfcheck confirmed that no reasoning tokens were produced. Sieve's components (LLMLingua-2 BERT-base, MiniLM, scikit-learn [54]) ran on the CPU. Token totals use the Qwen tokenizer with a chat-template overhead calibrated against the server; on a 1,821-token calibration prompt the tokenizer and the server agreed exactly (1,821 vs. 1,821).

C. Workload Generation

Conversations are generated from seed 7 with invented entities (codenames, people, figures), so the model cannot answer from prior knowledge. Filler text comes from a template grammar that contains no digits and no invented names; a unit test checks that each gold value appears only in the messages that the ground truth marks as needed. The benchmark has 76 conversations with 180 evaluation questions in eleven families (Table II). After each question is answered, the scripted gold answer is appended to the history, so a wrong answer never propagates and every condition sees the same history. Question types are numeric lookup (67), name lookup (64), decision recall (20), multi-hop (9), unanswerable (8), latest-value tracking (8), negation (4). 26% of questions (47 of 180) had a raw prompt longer than n_{ctx}. The router was trained on 456 further conversations from six seeds disjoint from the benchmark seed.

D. Conditions

Table III lists the conditions. C1 and C2 are matched per question to the prompt size C4 produced, so they compare strategies at an equal token budget. The five ablations each remove or replace one component of C4.

E. Protocol

Each condition ran as its own operating-system process and its own pass over all conversations; conditions were never interleaved, so no condition could inherit another's cache. The model was unloaded and reloaded before each pass. The first question of every conversation is *cold*, a novel prompt after a fresh load; later questions are *warm*. Each of the 2,160 model calls was logged with its prompt, response, per-segment decision trace and prompt diff.

TABLE III
EXPERIMENTAL CONDITIONS.

C0	Raw prompt (baseline), append-only history.
C1	Truncation: drop oldest whole messages to C4's token count.
C2	One uniform LLMLingua-2 rate on all unprotected segments, matched to C4's token count.
C3	Sieve with rules and the hand-written heuristic labeller; no classifier, no verifier.
C4	Full Sieve: rules, classifier, ledger/epochs, verifier, tools and prefetch.
C5	C4 without ledger or memo: re-routes and re-compresses every turn.
C6	Raw prompt with $n_{\text{ctx}} = 8192$: accuracy reference.
Ablations	C4 −verifier; C4 −prefetch (tool only); C4 −ledger; C4 with a heuristic-trained classifier; C4 with only the six core features.

TABLE IV
METRIC DEFINITIONS. T_{RAW}, T_{SENT}: TOKENIZER COUNTS OF THE RAW AND THE SENT PROMPT, INCLUDING CHAT TEMPLATE AND ALL SIEVE OVERHEAD.

Accuracy	Mean of a per-question score: normalized containment of a gold string; exact integer match for numbers; day, month and year all present for dates; first yes/no word; refusal pattern without numbers for unanswerable questions.
Tokens saved	$1 - T_{\text{sent}}/T_{\text{raw}}$, paired per question with C0.
Headroom	$n_{\text{ctx}} - T_{\text{sent}}$.
TTFT	Client time from sending the request to the first answer token, including tool round trips.
Prefix reuse	$1 - t_{\text{prefill}}/\hat{t}_{\text{cold}}(T_{\text{sent}})$, with $\hat{t}_{\text{cold}}(n) = an + bn^2$ fitted on novel prompts (Sec. IV-H).
Prefilled tokens	$T_{\text{sent}} \times (1 - \text{reuse})$, summed over a conversation.
Atom recall	Share of the question's gold atoms present in the sent prompt.
Sieve CPU	Segmentation, routing, compression, verification and assembly time per call.
Decode speed	Generated tokens / generation time; independent of the prompt, used as a machine-health indicator.

F. Metrics and How They Are Derived

Table IV defines every reported quantity and the logged field it is computed from.

G. Statistical Analysis

Every comparison is paired by question against C0. For a statistic θ of the per-question differences we report the observed value and a 95% percentile bootstrap interval from 2,000 resamples of questions [56]; medians are used for latency. A difference is called significant only when its interval excludes zero. The claims list is generated by the same code.

H. Measuring Prefix Reuse

On this server the reported `prompt_eval_count` equals the full prompt length even when the prefix comes from the cache, so it cannot reveal reuse. Prefill *time*, however, collapses on a cache hit. We therefore fit cold prefill time on novel prompts of 286, 545, 1,058, 1,841, 2,648, 3,435 tokens, obtaining $a = 1.699$ ms per token and $b = 0.0419$ µs per token^2, and estimate reuse as in Table IV. Section V-C validates the estimator.

I. Data Integrity Safeguards

Three issues arose during the campaign and were handled as follows. (i) A first full run was discarded after the host began

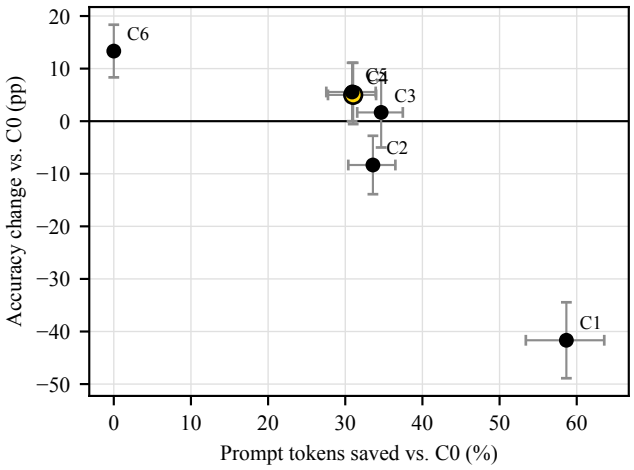


Fig. 2. Prompt-token savings versus accuracy change relative to C0, with 95% paired bootstrap intervals. C4 is the filled yellow marker. Derived from the paired per-question differences in the run log.

swapping memory, which depressed decode speed; every call now carries a flag when decode speed falls below half the calibrated value. (ii) During a network outage three passes silently fell back to an approximate tokenizer and, for two of them, to an extractive compressor; they were discarded and rerun with models loaded offline, and the benchmark now refuses to run with any degraded component. All 2,160 analysed calls ran with the exact tokenizer and LLMLingua-2 (0 fallbacks). (iii) One conversation of one pass was interrupted and re-run; the analysis keeps the last record per question. Residual machine-load variation is reported in Section V-B.

V. Results

A. RQ1: Tokens, Headroom and Accuracy

Tables V and VI answer RQ1. C4 sent 1,703 prompt tokens on average against 2,747 for C0, a paired reduction of 31.0% (95% CI 27.8–34.0%), and mean output headroom rose from 1,349 to 2,393 tokens. Accuracy was 0.911 for C4 and 0.861 for C0; the paired difference of +5.0 percentage points (CI -0.6 to +11.1) is not statistically distinguishable from zero. The gain is concentrated where the raw prompt overflowed the window and the server truncated it: on those 47 questions C4 changed accuracy by +36.2 pp (CI +19.1 to +53.2). The 8k-context reference C6 reached 0.994, which bounds what any prompt-shaping method can recover here. At a matched budget, truncation (C1) changed accuracy by -41.7 pp (CI -48.9 to -34.4) because it drops whole documents, and uniform compression (C2) by -8.3 pp (CI -13.9 to -2.8). Fig. 2 plots savings against accuracy for every condition.

B. RQ2: Latency

On cold questions C4 reduced median TTFT by 1.12 s (CI 0.66 to 1.49), because it prefills fewer tokens (Fig. 3). On warm questions most of the raw prompt is already cached and the medians were 0.33 s for C4 and 0.46 s for C0. Mean Sieve CPU time was 84 ms per call for C4. Decode speed does not

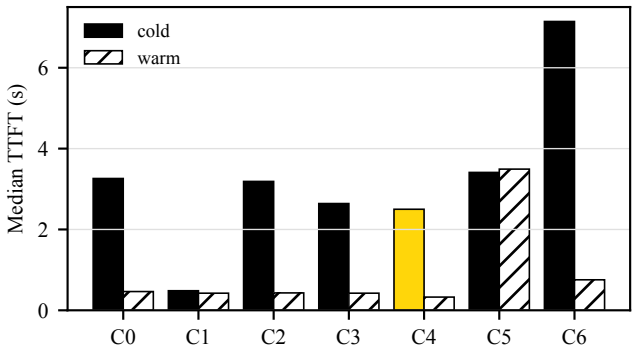


Fig. 3. Median time to first token for cold (first question of a conversation) and warm (later) questions per condition.

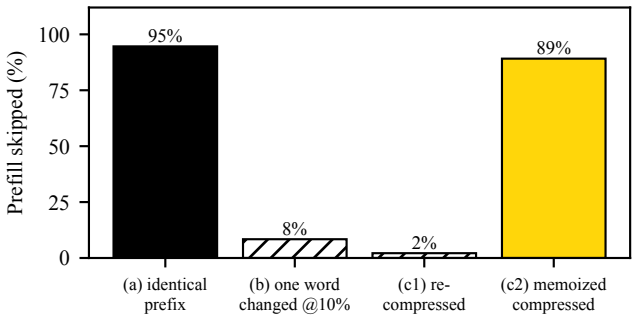


Fig. 4. Controlled cache test on the live server: share of the second request's prefill that the server skipped.

depend on the prompt, so it serves as a health indicator for the host. The passes C2 Uniform LLMLingua-2, C5 Sieve, no ledger, C6 8k context, C4 −verifier ran at a median decode speed below 75% of C0's 26.8 tok/s because of host memory pressure (calls flagged as degraded: C2 18, C5 11, C6 35, C4 16). Their absolute latencies are inflated and are not used for latency comparisons; their token and accuracy results are unaffected. C0 and C4 ran at matching decode speeds (26.8 and 28.9 tok/s).

C. RQ3: The Cache–Compression Conflict

Fig. 4 isolates the conflict with four request pairs on a 1,871-token prefix. With an identical prefix the server skipped 95% of prefill; changing a single word at 10% depth reduced reuse to 8%, close to the 10% that precedes the change, which validates the timing estimator. Re-compressing the prefix at a different ratio reduced reuse to 2%, and sending the memoized compressed string restored it to 89%. The benchmark shows the same effect at scale: C4 averaged 45% prefix reuse with 84 ms of CPU per call, while C5, which re-routes and re-compresses every turn, averaged 13% reuse and 2,022 ms of CPU per call. Summed over a conversation, C4 prefilled 1,981 tokens, C5 3,268 and C0 2,824.

D. RQ4: Ablations

Table VII removes one component at a time. Removing the verifier changed accuracy from 0.911 to 0.850 while saving

TABLE V
RESULTS PER CONDITION OVER 180 QUESTIONS FROM 76 CONVERSATIONS. ACCURACY WITH 95% BOOTSTRAP INTERVAL. PROMPT TOKENS ARE THOSE ACTUALLY SENT, INCLUDING ALL SIEVE OVERHEAD. TTFT MEDIANS IN SECONDS. REUSE IS THE TIMING-DERIVED SHARE OF PREFILL SKIPPED. DECODE IS THE PASS'S MEDIAN DECODE SPEED (MACHINE-HEALTH INDICATOR).

Condition	Accuracy [95% CI]	Prompt tok.	Saved %	Headroom	$\text{TTFT}_{\text{cold}}$	$\text{TTFT}_{\text{warm}}$	Reuse %	Sieve CPU (ms)	Decode (tok/s)
C0 Raw prompt	0.861 [0.81, 0.91]	2,747	0.0	1,349	3.26	0.46	47	0	26.8
C1 Truncate	0.444 [0.37, 0.52]	999	58.7	3,097	0.48	0.42	17	0	27.4
C2 Uniform LLMLingua-2	0.778 [0.72, 0.84]	1,652	33.6	2,444	3.19	0.43	34	957	17.4
C3 Sieve, rules	0.878 [0.83, 0.92]	1,646	34.7	2,450	2.64	0.42	44	69	28.7
C4 Sieve, full	0.911 [0.87, 0.95]	1,703	31.0	2,393	2.50	0.33	45	84	28.9
C5 Sieve, no ledger	0.917 [0.87, 0.96]	1,698	30.9	2,398	3.41	3.49	13	2,022	15.6
C6 8k context	0.994 [0.98, 1.00]	2,747	0.0	5,445	7.14	0.75	39	0	15.4

TABLE VI
PAIRED DIFFERENCES AGAINST C0: MEAN [95% BOOTSTRAP CI]. THE LAST COLUMN COVERS THE 47 QUESTIONS WHOSE RAW PROMPT EXCEEDED n_{CTX}.

	ΔAcc (pp)	Saved (%)	$\Delta\text{TTFT}_{\text{cold}}$ (s)	ΔAcc, overflow (pp)
C1	-41.7 [-48.9, -34.4]	58.7 [53.4, 63.6]	-1.36 [-2.32, -0.58]	-8.5 [-17.0, -2.1]
C2	-8.3 [-13.9, -2.8]	33.6 [30.4, 36.5]	-0.25 [-0.61, 0.05]	-4.3 [-19.2, 12.8]
C3	+1.7 [-5.0, 7.8]	34.7 [31.5, 37.5]	-1.15 [-1.47, -0.92]	+36.2 [21.3, 51.1]
C4	+5.0 [-0.6, 11.1]	31.0 [27.8, 34.0]	-1.12 [-1.49, -0.66]	+36.2 [19.1, 53.2]
C5	+5.6 [0.0, 11.1]	30.9 [27.5, 33.9]	-0.20 [-0.38, 0.15]	+34.0 [17.0, 51.1]
C6	+13.3 [8.3, 18.3]	0.0 [0.0, 0.0]	+2.14 [1.61, 2.80]	+51.1 [36.2, 66.0]

TABLE VII
ABLATIONS OF C4 OVER 180 QUESTIONS. PREFILL/CONV.: TOKENS PREFILLED PER CONVERSATION.

Variant	Acc.	ΔAcc vs C0 (pp)	Saved %	Reuse %	Prefill/conv.	Re-infl.
C4 Sieve, full	0.911	+5.0 [-0.6, 11.1]	31.0	45	1,981	270
C4 −verifier	0.850	-1.1 [-7.8, 5.6]	32.6	41	2,102	0
C4 −prefetch	0.911	+5.0 [-0.6, 11.1]	31.0	42	2,118	270
C4 −ledger	0.917	+5.6 [0.0, 11.1]	30.9	17	3,064	373
C4, heuristic-trained	0.917	+5.6 [0.0, 11.7]	32.6	43	2,049	268
C4, core-6 features	0.939	+7.8 [2.8, 13.3]	31.2	44	2,059	255

32.6% of tokens instead of 31.0%; with the verifier, C4 re-inflated 270 segment compressions. Removing the ledger lowered mean prefix reuse from 45% to 17% and raised prefill per conversation from 1,981 to 3,064 tokens. The classifier trained on heuristic labels reached 0.917 and the classifier restricted to the six core features 0.939, against 0.911 for C4; their paired intervals overlap with C4's. Prefetching never triggered in any condition, so the −prefetch variant sent the same prompts as C4.

E. RQ5: Router Accuracy and Externalization Break-Even

Table VIII scores the routers on conversations they never saw. The oracle-trained classifier reached macro-F1 0.949 with 79 dangerous errors, against 147 for the heuristic labeller; inference took 0.007 ms per segment. Because the oracle label is partly a fixed function of the features (age, size and repetition), a high F1 mainly shows that the labelling rule is learnable; the downstream comparison of C3 and C4 (Table V) is the more meaningful evidence. Fig. 5 sweeps the probability p that a question needs a large archived document that C4 externalizes (8 cold conversations per point). Mean net time per question was 4.47 s for C4 and 8.51 s for C0 at $p = 0$, and 7.63 s and 9.42 s at $p = 1$, where C4 expanded on 75% of questions. The curves did not cross in the swept range: in this setting externalization paid off even when every question needed the document, because C0's prompt overflowed the window and was prefilled in full on every call. Accuracy at $p = 1$ was 0.88 for C4 and 0.50 for C0.

TABLE VIII
ROUTERS ON HELD-OUT CONVERSATIONS (14,586 SEGMENT DECISIONS FROM 456 CONVERSATIONS, GROUPKFOLD BY CONVERSATION; GROUND TRUTH = ORACLE LABELS). DANGEROUS: A SEGMENT NEEDED BY A CURRENT OR FUTURE QUESTION SENT TO EXTERNALIZE (OUT OF 1,897).

Router	Macro-F1	Ext. P / R	Ratio MAE	Dangerous
Hard rules only	0.642	0.00 / 0.00	0.367	0
Heuristic labeller (C3)	0.912	0.73 / 0.85	0.052	147
GBDT, heuristic labels	0.910	0.73 / 0.85	0.055	146
GBDT, oracle labels (C4)	0.949	0.94 / 0.83	0.023	79
GBDT, oracle, core-6	0.943	0.90 / 0.84	0.044	137

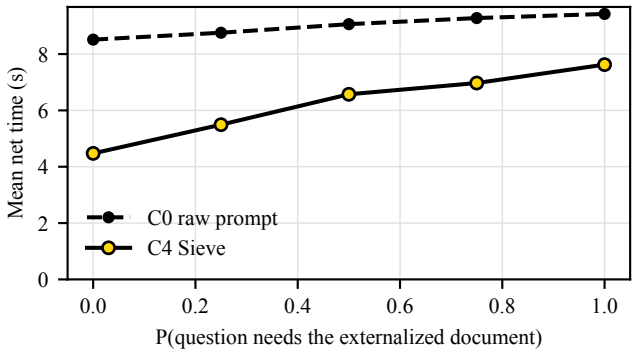


Fig. 5. Externalization break-even: mean net time per question (Sieve CPU time plus model time, including tool round trips) as the probability that a question needs the externalized document grows.

F. Breakdown by Workload

Table IX breaks accuracy down by family. Groups are small, so single rows are indicative only. C4 was above C0 in families B, C, E, F, G and below it in D, J, K.

VI. DISCUSSION

What the results support. Sieve reduced prompt tokens by 31.0% with an interval that excludes zero, improved accuracy on prompts that overflowed the window (+36.2 pp), lowered median cold TTFT by 1.12 s; its overall accuracy change is not statistically distinguishable from zero. The cache test and the C4–C5 comparison support the ledger as the mechanism that keeps compression compatible with prefix caching.

TABLE IX
ACCURACY BY WORKLOAD FAMILY (BOLD: ABOVE C0 ON THE SAME QUESTIONS).

Fam.	n	C0	C1	C2	C3	C4	C5	C6
A	19	0.84	0.84	0.84	0.84	0.84	0.84	**1.00**
B	20	0.90	0.65	0.90	**1.00**	**1.00**	**0.95**	**0.95**
C	27	0.74	0.00	**0.78**	**0.81**	**1.00**	**1.00**	**1.00**
D	18	1.00	0.17	0.56	0.89	0.78	1.00	1.00
E	10	0.80	0.20	0.70	**0.90**	**1.00**	**1.00**	**1.00**
F	16	0.94	0.50	0.81	0.94	**1.00**	**1.00**	**1.00**
G	16	0.62	0.00	0.62	**0.94**	**0.88**	**0.88**	**1.00**
H	6	0.83	0.00	0.67	**1.00**	0.83	0.83	**1.00**
I	8	0.75	0.50	**0.88**	0.50	0.75	0.62	**1.00**
J	8	0.88	0.88	0.88	0.62	0.75	0.88	**1.00**
K	32	1.00	0.84	0.84	0.94	0.94	0.88	1.00

Where Sieve does not help. On short prompts (under about 1,500 tokens) C4 saved 15.8% of tokens and reached accuracy 0.94 against 1.00 for C0. Families D, J, K lost accuracy under C4. On warm questions the raw prompt's prefix is already cached, so latency differences there are small (0.33 s vs. 0.46 s median). C4 hard-truncated 15 prompts that remained over budget after compression and externalization.

Tool use without reasoning. With reasoning disabled, the model called `expand_context` only when prompted by the tail hint; C4 expanded on 5% of questions, and 4 needed segments were stubbed but never expanded. Prefetching never triggered at the 0.75 similarity threshold, so the −prefetch ablation sent the same content as C4 and differs only in timing.

Threats to validity. Internal: latency depends on machine state; we ran conditions in separate passes, flagged slow calls and report decode speed per pass. *Construct:* prefix reuse is estimated from a fitted cold-prefill curve rather than read from the server. *External:* the workloads are synthetic with invented entities, the benchmark is small, and it covers one model on one machine with no CPU offload; results describe this suite rather than real conversations. Stub summaries are extractive and only text attachments are supported.

VII. Conclusion

Sieve routes each segment of a prompt to compression, caching or externalization and keeps the emitted text byte-stable with a ledger. On 180 ground-truth questions with Qwen3-14B and a 4096-token window, it reduced prompt tokens by 31.0% and raised output headroom from 1,349 to 2,393 tokens, with accuracy statistically indistinguishable from the raw prompt; on questions whose raw prompt overflowed the window, accuracy rose by 36.2 percentage points A controlled test showed that re-compressing a prefix destroys prefix reuse and that byte-stable memoized output restores it. Future work includes evaluating real conversation logs, other model families and servers, abstractive stub summaries, and learning when to expand rather than prompting for it.

Code and Data Availability

The implementation, configuration, benchmark generator, test suite, all 2,160 logged calls with per-segment traces, and the scripts that regenerate every table and figure in this paper are available at https://github.com/Tarun-Vaibhav-V/sieve [17].

References

[1] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile edge intelligence for large language models: A contemporary survey," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 6, p. 3820–3860, Dec. 2025. [Online]. Available: http://dx.doi.org/10.1109/comst.2025.3527641

[2] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green ai," *Communications of the ACM*, vol. 63, no. 12, p. 54–63, Nov. 2020. [Online]. Available: http://dx.doi.org/10.1145/3381831

[3] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 5, p. 1–72, Aug. 2025. [Online]. Available: http://dx.doi.org/10.1145/3744746

[4] M. A. K. Raiaan, M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam, "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," *IEEE Access*, vol. 12, p. 26839–26874, 2024. [Online]. Available: http://dx.doi.org/10.1109/access.2024.3365742

[5] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, p. 1–79, Nov. 2024. [Online]. Available: http://dx.doi.org/10.1145/3695988

[6] Y. Li, B. Dong, F. Guerin, and C. Lin, "Compressing Context to Enhance Inference Efficiency of Large Language Models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 6342–6353.

[7] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 13 358–13 376.

[8] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 1658–1677.

[9] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 963–981.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. ACM, Oct. 2023, p. 611–626. [Online]. Available: http://dx.doi.org/10.1145/3600006.3613165

[11] I. Gim, G. Chen, S. seob Lee, N. Sarda, A. Khandelwal, and L. Zhong, "Prompt Cache: Modular Attention Reuse for Low-Latency Inference," arXiv:2311.04934, 2023.

[12] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," arXiv:2312.07104, 2023.

[13] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[14] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model-based agents," *ACM Transactions on Information Systems*, vol. 43, no. 6, p. 1–47, 2025. [Online]. Available: http://dx.doi.org/10.1145/3748302

[15] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv:2302.04761, 2023.

[16] Y. Qin, S. Hu, Y. Lin, W. Chen, N. Ding, G. Cui, Z. Zeng, X. Zhou, Y. Huang, C. Xiao, C. Han, Y. R. Fung, Y. Su, H. Wang, C. Qian, R. Tian, K. Zhu, S. Liang, X. Shen, B. Xu, Z. Zhang, Y. Ye, B. Li, Z. Tang, J. Yi, Y. Zhu, Z. Dai, L. Yan, X. Cong, Y. Lu, W. Zhao, Y. Huang, J. Yan, X. Han, X. Sun, D. Li, J. Phang, C. Yang, T. Wu, H. Ji, G. Li, Z. Liu, and M. Sun, "Tool learning with foundation

models," *ACM Computing Surveys*, vol. 57, no. 4, p. 1–40, Dec. 2024. [Online]. Available: http://dx.doi.org/10.1145/3704435

[17] [Author Name], "Sieve: segment-level prompt routing for local LLMs — code, logs and per-call traces," https://github.com/Tarun-Vaibhav-V/sieve, 2026.

[18] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *Science China Technological Sciences*, vol. 63, no. 10, p. 1872–1897, 2020. [Online]. Available: http://dx.doi.org/10.1007/s11431-020-1647-3

[19] X. Han, Z. Zhang, N. Ding, Y. Gu, X. Liu, Y. Huo, J. Qiu, Y. Yao, A. Zhang, L. Zhang, W. Han, M. Huang, Q. Jin, Y. Lan, Y. Liu, Z. Liu, Z. Lu, X. Qiu, R. Song, J. Tang, J.-R. Wen, J. Yuan, W. X. Zhao, and J. Zhu, "Pre-trained models: Past, present and future," *AI Open*, vol. 2, p. 225–250, 2021. [Online]. Available: http://dx.doi.org/10.1016/j.aiopen.2021.08.002

[20] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Computing Surveys*, vol. 56, no. 2, p. 1–40, 2023. [Online]. Available: http://dx.doi.org/10.1145/3605943

[21] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, and X. Hu, "Harnessing the power of llms in practice: A survey on chatgpt and beyond," *ACM Transactions on Knowledge Discovery from Data*, vol. 18, no. 6, p. 1–32, Apr. 2024. [Online]. Available: http://dx.doi.org/10.1145/3649506

[22] T. Lin, Y. Wang, X. Liu, and X. Qiu, "A survey of transformers," *AI Open*, vol. 3, p. 111–132, 2022. [Online]. Available: http://dx.doi.org/10.1016/j.aiopen.2022.10.001

[23] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Computing Surveys*, vol. 55, no. 9, p. 1–35, Jan. 2023. [Online]. Available: http://dx.doi.org/10.1145/3560815

[24] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering for large language models," *Patterns*, vol. 6, no. 6, p. 101260, 2025. [Online]. Available: http://dx.doi.org/10.1016/j.patter.2025.101260

[25] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, p. 1–45, Mar. 2024. [Online]. Available: http://dx.doi.org/10.1145/3641289

[26] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu *et al.*, "Qwen3 Technical Report," arXiv:2505.09388, 2025.

[27] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, "Efficient transformers: A survey," *ACM Computing Surveys*, vol. 55, no. 6, p. 1–28, Dec. 2022. [Online]. Available: http://dx.doi.org/10.1145/3530811

[28] Q. Fournier, G. M. Caron, and D. Aloise, "A practical survey on faster and lighter transformers," *ACM Computing Surveys*, vol. 55, no. 14s, p. 1–40, 2023. [Online]. Available: http://dx.doi.org/10.1145/3586074

[29] M. Treviso, J.-U. Lee, T. Ji, B. v. Aken, Q. Cao, M. R. Ciosici, M. Hassid, K. Heafield, S. Hooker, C. Raffel, P. H. Martins, A. F. T. Martins, J. Z. Forde, P. Milder, E. Simpson, N. Slonim, J. Dodge, E. Strubell, N. Balasubramanian, L. Derczynski, I. Gurevych, and R. Schwartz, "Efficient methods for natural language processing: A survey," *Transactions of the Association for Computational Linguistics*, vol. 11, p. 826–860, 2023. [Online]. Available: http://dx.doi.org/10.1162/tacl_a_00577

[30] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," *Transactions of the Association for Computational Linguistics*, vol. 12, p. 1556–1577, 2024. [Online]. Available: http://dx.doi.org/10.1162/tacl_a_00704

[31] B. L. Deng, G. Li, S. Han, L. Shi, and Y. Xie, "Model compression and hardware acceleration for neural networks: A comprehensive survey," *Proceedings of the IEEE*, vol. 108, no. 4, p. 485–532, Apr. 2020. [Online]. Available: http://dx.doi.org/10.1109/jproc.2020.2976475

[32] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, p. 10558–10578, Dec. 2024. [Online]. Available: http://dx.doi.org/10.1109/tpami.2024.3447085

[33] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *International Journal of Computer Vision*, vol. 129, no. 6, p. 1789–1819, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1007/s11263-021-01453-z

[34] H. Jung and K.-J. Kim, "Discrete prompt compression with reinforcement learning," *IEEE Access*, vol. 12, p. 72578–72587, 2024. [Online]. Available: http://dx.doi.org/10.1109/access.2024.3403426

[35] J. Mu, X. L. Li, and N. Goodman, "Learning to Compress Prompts with Gist Tokens," arXiv:2304.08467, 2023.

[36] A. Chevalier, A. Wettig, A. Ajith, and D. Chen, "Adapting Language Models to Compress Contexts," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 3829–3846.

[37] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation," arXiv:2310.04408, 2023.

[38] W. S. El-Kassas, C. R. Salama, A. A. Rafea, and H. K. Mohamed, "Automatic text summarization: A comprehensive survey," *Expert Systems with Applications*, vol. 165, p. 113679, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1016/j.eswa.2020.113679

[39] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, H. Jin, T. Chen, and Z. Jia, "Towards efficient generative large language model serving: A survey from algorithms to systems," *ACM Computing Surveys*, vol. 58, no. 1, p. 1–37, 2025. [Online]. Available: http://dx.doi.org/10.1145/3754448

[40] ggml-org contributors, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp, accessed Oct. 2026.

[41] Ollama contributors, "Ollama," https://github.com/ollama/ollama, version 0.32.15. Accessed Oct. 2026.

[42] Y. Huang and J. X. Huang, "A survey on retrieval-augmented text generation for large language models," *ACM Computing Surveys*, vol. 58, no. 12, p. 1–38, May 2026. [Online]. Available: http://dx.doi.org/10.1145/3805774

[43] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, and J.-R. Wen, "Large language models for information retrieval: A survey," *ACM Transactions on Information Systems*, vol. 44, no. 1, p. 1–54, Nov. 2025. [Online]. Available: http://dx.doi.org/10.1145/3748304

[44] W. X. Zhao, J. Liu, R. Ren, and J.-R. Wen, "Dense text retrieval based on pretrained language models: A survey," *ACM Transactions on Information Systems*, vol. 42, no. 4, p. 1–60, Feb. 2024. [Online]. Available: http://dx.doi.org/10.1145/3637870

[45] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[46] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," arXiv:2002.10957, 2020.

[47] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, Mar. 2024. [Online]. Available: http://dx.doi.org/10.1007/s11704-024-40231-1

[48] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Qin, Y. Zheng, X. Qiu, X. Huang, Q. Zhang, and T. Gui, "The rise and potential of large language model based agents: a survey," *Science China Information Sciences*, vol. 68, no. 2, Jan. 2025. [Online]. Available: http://dx.doi.org/10.1007/s11432-024-4222-0

[49] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, p. 157–173, 2024. [Online]. Available: http://dx.doi.org/10.1162/tacl_a_00638

[50] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, p. 1–38, Mar. 2023. [Online]. Available: http://dx.doi.org/10.1145/3571730

[51] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, p. 1–55, Jan. 2025. [Online]. Available: http://dx.doi.org/10.1145/3703155

[52] J. H. Friedman, "Greedy function approximation: A gradient boosting machine." *The Annals of Statistics*, vol. 29, no. 5, Oct. 2001. [Online]. Available: http://dx.doi.org/10.1214/aos/1013203451

[53] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, p. 5–32, Oct. 2001. [Online]. Available: http://dx.doi.org/10.1023/a:1010933404324

[54] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[55] H. He and E. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, p. 1263–1284, 2009. [Online]. Available: http://dx.doi.org/10.1109/tkde.2008.239

[56] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, Jan. 1979. [Online]. Available: http://dx.doi.org/10.1214/aos/1176344552

[57] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, p. 1895–1923, Oct. 1998. [Online]. Available: http://dx.doi.org/10.1162/089976698300017197

[Author Name] [Short biography: degree, institution, research interests.]